React JS សម្រាប់អ្នកចាប់ផ្តើម ដល់កម្រិតមធ្យម

វគ្គសិក្សាពេញលេញ ១៧ មេរៀន ជាមួយលំហាត់អនុវត្តគ្រប់មេរៀន

ទិដ្ឋភាពទូទៅនៃវគ្គសិក្សា

- វគ្គសិក្សានេះរចនាឡើងសម្រាប់អ្នកមិនទាន់ចេះ React ពីដំបូង រហូតដល់អាចបង្កើតកម្មវិធីកម្រិតមធ្យមបាន
- មាន **១៧ មេរៀន** ផ្សារភ្ជាប់គ្នាជាជំហានៗ
- រាល់មេរៀនមាន **ខ្លឹមសារដែលបែងចែកជា Main Point និង Sub Point + លំហាត់អនុវត្ត**
- រយៈពេលប៉ាន់ស្មាន សរុបប្រហែល **២០-២២ ម៉ោង**
- តម្រូវការមុនចូលរៀន៖ ស្គាល់ HTML, CSS, និង JavaScript ជាមូលដ្ឋាន

មាតិការវគ្គសិក្សា (Course Outline)

១. ការណែនាំអំពី React JS ២. រៀបចំបរិស្ថានសម្រាប់អភិវឌ្ឍ React ៣. JSX - ការសរសេរ HTML ក្នុង JavaScript ៤. Components - សមាសភាគជាមូលដ្ឋាន ៥. Styling Components - ការតុបតែង Component ៦. Props - ការបញ្ជូនទិន្នន័យទៅ Component ៧. State - ការគ្រប់គ្រងទិន្នន័យជាមួយ useState ៨. ការគ្រប់គ្រងព្រឹត្តិការណ៍ (Event Handling) ៩. Conditional Rendering

---

មាតិការវគ្គសិក្សា (ត)

១០. ការបង្ហាញ Lists និង Keys ១១. Forms និង Controlled Inputs ១២. useEffect Hook - ការគ្រប់គ្រង Side Effect ១៣. Ref និង Portal ១៤. ការទាញទិន្នន័យពី API (Fetching Data) ១៥. Lifting State Up - ការទាក់ទងគ្នារវាង Components ១៦. React Router - ការបង្កើតទំព័រច្រើន (Navigation) ១៧. Custom Hooks និងគម្រោងបញ្ចប់វគ្គសិក្សា

មេរៀនទី១

ការណែនាំអំពី React JS

(រយៈពេលប៉ាន់ស្មាន ៤៥ នាទី)

១. React JS គឺជាអ្វី ១.១. ជា JavaScript Library សម្រាប់បង្កើត User Interface (UI) ១.២. បង្កើត និងថែទាំដោយក្រុមហ៊ុន Meta (Facebook) ២. ហេតុអ្វីត្រូវប្រើ React ២.១. Component-Based - បំបែក UI ជាផ្នែកតូចៗ ប្រើឡើងវិញបាន ២.២. Virtual DOM - ធ្វើឱ្យការ Update ទំព័របានលឿន និងរលូន ៣.

កន្លែងដែល React ត្រូវបានគេប្រើប្រាស់ ៣.១. Facebook, Instagram, Netflix, Airbnb ៤. SPA (Single Page Application) ខុសពីគេហទំព័រធម្មតាដូចម្តេច

លំហាត់អនុវត្ត

- រាយឈ្មោះកម្មវិធី ឬគេហទំព័រ ៣ ដែលអ្នកដឹងថាប្រើ React
- សរសេរពន្យល់ខ្លីៗ ៣-៤ ប្រយោគ អំពីហេតុអ្វី React ត្រូវបានគេនិយមប្រើច្រើនបច្ចុប្បន្ននេះ

មេរៀនទី២

រៀបចំបរិស្ថានសម្រាប់អភិវឌ្ឍ React

(រយៈពេលប៉ាន់ស្មាន ៦០ នាទី)

១. ការដំឡើងកម្មវិធីចាំបាច់ ១.១. Node.js ១.២. npm (Node Package Manager) ២. ការបង្កើតគម្រោង React ថ្មី ២.១. ប្រើ Vite (`npm create vite@latest`) ២.២. ជ្រើសរើស Template (React + JavaScript) ៣. រចនាសម្ព័ន្ធ Folder របស់គម្រោង ៣.១. src/, public/ ៣.២. App.jsx, main.jsx, package.json ៤. ការដំណើរការ Development Server (`npm run dev`) ៥. ការប្រើ VS Code និង Extension ដែលជួយសរសេរ React

លំហាត់អនុវត្ត

- បង្កើតគម្រោង React ថ្មីមួយឈ្មោះ `my-first-app` ដោយប្រើ Vite
- ដំណើរការគម្រោងនោះ ហើយថតរូប Screenshot ទំព័រដែលបង្ហាញលើ Browser

---

មេរៀនទី៣

JSX - ការសរសេរ HTML ក្នុង JavaScript

(រយៈពេលប៉ាន់ស្មាន ៦០ នាទី)

១. JSX គឺជាអ្វី ១.១. Syntax Extension របស់ JavaScript ១.២. ខុសពី HTML ពិតប្រាកដ ត្រូវបាន Compile ទៅជា JavaScript ២. ការប្រើ Expression ក្នុង JSX ២.១. ប្រើ Curly Braces `{ }` ដើម្បីបញ្ចូល JavaScript ៣. ច្បាប់សំខាន់ៗរបស់ JSX ៣.១. ត្រូវ Return Element តែមួយ (ប្រើ `<div>` ឬ `<>...</>`) ៣.២. ប្រើ `className` ជំនួសឱ្យ `class` ៣.៣. Tag ត្រូវបិទរាល់ពេល (`<img />`, `<br />`)

លំហាត់អនុវត្ត

- សរសេរ JSX មួយដើម្បីបង្ហាញព័ត៌មានផ្ទាល់ខ្លួន៖ ឈ្មោះ អាយុ និងវិជ្ជាជីវៈ
- ប្រើ Variable ៣ ខ្នាត រួចបង្ហាញតម្លៃរបស់វាដោយប្រើ `{ }` ក្នុង JSX

មេរៀនទី៤

Components - សមាសភាគជាមូលដ្ឋាន

(រយៈពេលជាំន់ស្ថាន ៦០ នាទី)

១. Component គឺជាអ្វី ១.១. Function ឬ Class ដែល Return JSX ១.២. ប្រើជា Building Block របស់ UI ២. ប្រភេទ Component ២.១. Functional Component (ទម្រង់ដែលគេប្រើច្រើនបំផុតបច្ចុប្បន្ននេះ) ៣. ការបង្កើត និងហៅប្រើ Component ៣.១. ការហៅប្រើក្នុង Component ផ្សេង (`<MyComponent />`) ៤. ការបំបែក UI ធំមួយ ទៅជា Component តូចៗ ៤.១. File មួយ Component មួយ ជាទម្លាប់ល្អ (Header.jsx, Footer.jsx)

លំហាត់អនុវត្ត

- បង្កើត Component ឈ្មោះ `StudentCard` សម្រាប់បង្ហាញព័ត៌មានសិស្សម្នាក់ (ឈ្មោះ, ថ្នាក់, សាលា)
- ហៅប្រើ `StudentCard` នោះចំនួន ៣ ដងក្នុង `App.jsx`

មេរៀនទី៥

Styling Components - ការតុបតែង Component

(រយៈពេលជាំន់ស្ថាន ៦០ នាទី)

១. វិធីសម្រាប់ Style Component ក្នុង React ១.១. CSS ធម្មតា (Plain CSS File) ១.២. Inline Style ១.៣. CSS Modules ១.៤. Library ដូចជា Tailwind CSS, Styled-components (ស្គាល់ប្រាថ្នា) ២. ការប្រើ Plain CSS File ២.១. ការ Import CSS File ចូល Component ២.២. ការប្រើ `className` ភ្ជាប់ Class CSS ៣. ការប្រើ Inline Style ៣.១. Style Object ត្រូវសរសេរជា camelCase (`backgroundColor`, `fontSize`) ៤. ការប្រើ CSS Modules ៤.១. File ឈ្មោះ `Component.module.css` ៤.២. ប្រយោជន៍៖ Class Name មិនប៉ះទង្គិចគ្នារវាង Component ផ្សេងៗ

លំហាត់អនុវត្ត

- បង្កើត Component `Card` បង្ហាញព័ត៌មានផលិតផល ហើយ Style វាដោយប្រើ **CSS Modules** ឱ្យមាន padding, border-radius និង box-shadow

---

មេរៀនទី៦

Props - ការបញ្ជូនទិន្នន័យទៅ Component

(រយៈពេលជាន់ស្មាន ៦០ នាទី)

១. Props គឺជាអ្វី ១.១. ប្រើដើម្បីបញ្ជូនទិន្នន័យពី Component មេ (Parent) ទៅកូន (Child) ២. របៀបបញ្ជូន និងទទួល Props ២.១. ការបញ្ជូន Props៖ `<UserCard name="សុខា" role="Developer" />` ២.២. ការទទួល Props ក្នុង Component តាមរយៈ Parameter ៣. Default Props ៣.១. កំណត់តម្លៃលំនាំដើម ករណីមិនបញ្ជូន Props ៤. Children Props ៤.១. ការប្រើ `props.children` ដើម្បីបញ្ជូន Content ចូលក្នុង Component

លំហាត់អនុវត្ត

- បង្កើត Component `UserCard` ដែលទទួល Props ៣ យ៉ាង៖ រូបភាព, ឈ្មោះ, និងតួនាទី
- ហៅប្រើ `UserCard` ចំនួន ៤ ដង ដោយផ្តល់ Props ខុសៗគ្នា

មេរៀនទី៧

State - ការគ្រប់គ្រងទិន្នន័យជាមួយ useState

(រយៈពេលជាន់ស្មាន ៩០ នាទី)

១. State គឺជាអ្វី ១.១. ទិន្នន័យដែលអាចផ្លាស់ប្តូរបាន ហើយធ្វើឱ្យ Component Render ឡើងវិញ ១.២. ភាពខុសគ្នារវាង Props និង State ២. ការប្រើ useState Hook ២.១. Syntax៖ `const [count, setCount] = useState(0)` ៣. ការ Update State ៣.១. ត្រូវប្រើ Function ដែលទទួលបានពី `useState` (មិនកែដោយផ្ទាល់) ៤. State ប្រភេទផ្សេងៗ ៤.១. Number, String ៤.២. Array, Object

លំហាត់អនុវត្ត

- បង្កើតកម្មវិធី **Counter App** មាន ៣ ប៊ូតុង៖ បន្ថែម (+), ដក (-), និងកំណត់ត្រឡប់ទៅសូន្យ (Reset)

មេរៀនទី៨

ការគ្រប់គ្រងព្រឹត្តិការណ៍ (Event Handling)

(រយៈពេលជាន់ស្មាន ៦០ នាទី)

១. Event ក្នុង React ១.១. `onClick`, `onChange`, `onSubmit`, `onMouseOver` ២. ការបញ្ជូន Function ទៅ Event ២.១. `<button onClick={handleClick}>ចុច</button>` ៣. ការប្រើ Event Object ៣.១.

`e.target.value` ៤. ការបញ្ជូន Argument ទៅ Handler Function ៤.១. ប្រើ Arrow Function (`onClick={() => handleClick(id)}`)

លំហាត់អនុវត្ត

- បង្កើតប៊ូតុង **Like** ដែលផ្លាស់ប្តូរពណ៌ និងអក្សរ (Like / Liked) ពេលចុច
- បន្ថែម Counter បង្ហាញចំនួនដងដែលអ្នកប្រើបានចុចប៊ូតុង

មេរៀនទី៩

Conditional Rendering

(រយៈពេលប៉ាន់ស្មាន ៤៥ នាទី)

១. ហេតុអ្វីត្រូវការ Conditional Rendering ១.១. ការបង្ហាញ JSX ខុសគ្នាទៅតាមលក្ខខណ្ឌ (Condition) ២. វិធីសរសេរ Condition ក្នុង React ២.១. `if/else` ធម្មតាក្នុង Function មុន Return ២.២. Ternary Operator (`condition ? A : B`) ២.៣. Logical `&&` Operator ៣. ការលាក់ / បង្ហាញ Element អាស្រ័យលើ State

លំហាត់អនុវត្ត

- បង្កើតប្រព័ន្ធ **Login/Logout**៖ បង្ហាញប៊ូតុង "ចូល" ពេលមិនទាន់ Login និងបង្ហាញ "ចេញ" ពេល Login រួច
- បន្ថែមអក្សរសួរស្តី "សូមស្វាគមន៍" ពេលអ្នកប្រើ Login ជោគជ័យ

មេរៀនទី១០

ការបង្ហាញ Lists និង Keys

(រយៈពេលប៉ាន់ស្មាន ៦០ នាទី)

១. ការប្រើ `.map()` ដើម្បីបង្ហាញ List ១.១. ការបង្ហាញទិន្នន័យជា List ពី Array ២. Key គឺជាអ្វី ហើយហេតុអ្វីត្រូវការ ២.១. ជួយ React តាមដាន Element ណាមួយផ្លាស់ប្តូរ ៣. ការបង្ហាញ Array of Objects ជា List Component ៤. កំហុសញឹកញាប់ ៤.១. ប្រើ `index` ជា Key ដោយមិនយកចិត្តទុកដាក់ ពេល List ផ្លាស់ប្តូរលំដាប់

លំហាត់អនុវត្ត

- បង្កើត Array ផលិតផល (Product) មាន ឈ្មោះ និងតម្លៃ យ៉ាងតិច ៥ប្រភេទ
- បង្ហាញវាជា **Product List** ដោយប្រើ `.map()` ជាមួយ `key` ត្រឹមត្រូវ

មេរៀនទី១១

Forms និង Controlled Inputs

(រយៈពេលជាន់ស្មាន ៩០ នាទី)

១. Controlled vs Uncontrolled Input ១.១. ភាពខុសគ្នារវាង Input ដែល React គ្រប់គ្រង និងមិនគ្រប់គ្រង ២. ការប្រើ useState គ្រប់គ្រងតម្លៃ Input ២.១. `value` + `onChange` ៣. ការគ្រប់គ្រង Form ដែលមាន Input ច្រើនវាល ៣.១. ប្រើ State ជា Object តែមួយ ៤. ការ Validate ទិន្នន័យបឋម ៤.១. ពិនិត្យប្រអប់ទទេ មុន Submit ៥. ការប្រើ `onSubmit` និង `e.preventDefault()`

លំហាត់អនុវត្ត

- បង្កើត **Form ចុះឈ្មោះ** មាន ៣ វាលៈ ឈ្មោះ, អ៊ីម៉ែល, ពាក្យសម្ងាត់
- ពេលចុច Submit ឱ្យបង្ហាញទិន្នន័យដែលបានបញ្ចូលនៅផ្នែកខាងក្រោម Form

---

មេរៀនទី១២

useEffect Hook - ការគ្រប់គ្រង Side Effect

(រយៈពេលជាន់ស្មាន ៩០ នាទី)

១. useEffect គឺជាអ្វី ១.១. ដំណើរការ Code ដោយស្វ័យប្រវត្តិ ពេល Component Render ឬ State ផ្លាស់ប្តូរ ១.២. Syntax៖ `useEffect(() => { ... }, [dependencies])` ២. Dependency Array ២.១. `[]` = ដំណើរការតែម្តងគត់ ពេល Component ដំបូង Mount ២.២. `[value]` = ដំណើរការពេល `value` ផ្លាស់ប្តូរ ២.៣. គ្មាន Array = ដំណើរការគ្រប់ពេល Render ៣. Cleanup Function ៣.១. ប្រើដើម្បីបញ្ឈប់ Timer ឬ Subscription ៤. ការប្រើ useEffect ហៅ API នៅពេល Component Mount

លំហាត់អនុវត្ត

- បង្កើតកម្មវិធីបង្ហាញ **ម៉ោងបច្ចុប្បន្ន** ដែល Update រាល់ ១ វិនាទី ដោយប្រើ useEffect + setInterval
- ត្រូវប្រើ Cleanup Function ដើម្បីបញ្ឈប់ Timer ពេល Component ត្រូវលុប

មេរៀនទី១៣

Ref និង Portal

(រយៈពេលជាន់ស្មាន ៩០ នាទី)

១. useRef Hook គឺជាអ្វី ១.១. ប្រើសម្រាប់ចូលដំណើរការ DOM Element ដោយផ្ទាល់ ១.២. ខុសពី State៖ ការប្តូរតម្លៃ Ref មិនធ្វើឱ្យ Component Render ឡើងវិញ ២. ករណីប្រើ useRef ២.១. Focus លើ Input ដោយស្វ័យប្រវត្តិ ២.២. រក្សាតម្លៃដែលមិនត្រូវការ Re-render (ឧ. Timer ID) ៣. Portal គឺជាអ្វី ៣.១. ប្រើ `createPortal` ដើម្បី Render Component ទៅខាងក្រៅ DOM Tree ធម្មតា ៣.២. ប្រើញឹកញាប់សម្រាប់ Modal, Tooltip, Popup ៤. ការប្រើ `createPortal` ជាមួយ Modal Component

លំហាត់អនុវត្ត

- បង្កើត **Modal Popup** ដោយប្រើ Portal
- ប្រើ `useRef` ដើម្បី Focus លើ Input ដំបូងភ្លាមៗ ពេល Modal បើក

មេរៀនទី១៤

ការទាញទិន្នន័យពី API (Fetching Data)

(រយៈពេលជាន់ស្មាន ៩០ នាទី)

១. ការប្រើ Fetch API ឬ Axios ១.១. ការទាញទិន្នន័យពី Server ២. ការផ្សំ `useEffect` + `useState` ២.១. ហៅ API ហើយរក្សាទុកលទ្ធផលក្នុង State ៣. ការគ្រប់គ្រង Loading State និង Error State ៣.១. Loading State (កំពុងផ្ទុក) ៣.២. Error State (ករណីបរាជ័យ) ៤. ការបង្ហាញទិន្នន័យជា List ពេលទាញបានជោគជ័យ

លំហាត់អនុវត្ត

- ប្រើ API សាធារណៈ: `https://jsonplaceholder.typicode.com/users`
- បង្កើតកម្មវិធីបង្ហាញ **User List** ដោយមាន Loading Message ខណៈកំពុងផ្ទុកទិន្នន័យ

មេរៀនទី១៥

Lifting State Up - ការទាក់ទងគ្នារវាង Components

(រយៈពេលជាន់ស្មាន ៩០ នាទី)

១. Lifting State Up គឺជាអ្វី ១.១. ការផ្ទុក State នៅ Component មេ ដើម្បីចែករំលែកជាមួយ Component កូនច្រើន ២. ការបញ្ជូន Function ពី Parent ទៅ Child ២.១. ឱ្យ Child អាចកែប្រែ State របស់ Parent ៣. ការផ្លាស់ប្តូរទិន្នន័យរវាង Sibling Components ៣.១. តាមរយៈ Parent រួម ៤. គោលការណ៍ផ្ទុក State ៤.១. "State គួររក្សានៅកន្លែងណាដែលប្រើប្រាស់រួមគ្នាបំផុត"

លំហាត់អនុវត្ត

- បង្កើតកម្មវិធី **Todo List** ដែលអាចៈ

- បញ្ចូលកិច្ចការថ្មី (Add)
- លុបកិច្ចការ (Delete)
- សម្គាល់ថារួចរាល់ (Mark as Done)

មេរៀនទី១៦

React Router - ការបង្កើតទំព័រច្រើន

(រយៈពេលជាន់ស្មាន ៩០ នាទី)

១. ហេតុអ្វីត្រូវការ React Router ១.១. ក្នុង Single Page Application ២. ការដំឡើង Library `react-router-dom` ៣. សមាសភាគសំខាន់ៗ ៣.១. `<BrowserRouter>`, `<Routes>`, `<Route>`, `<Link>` ៤. ការប្តូរទំព័រដោយកម្មវិធី ៤.១. ប្រើ `useNavigate` (Programmatically) ៥. Dynamic Route Params ៥.១. ឧទាហរណ៍៖ `/product/:id`

លំហាត់អនុវត្ត

- បង្កើតគេហទំព័រមាន ៣ ទំព័រ៖ **Home, About, Contact**
- ប្រើ `<Link>` ដើម្បីប្តូររវាងទំព័រដោយមិន Reload ទំព័រទាំងមូល

មេរៀនទី១៧

Custom Hooks និងគម្រោងបញ្ចប់វគ្គសិក្សា

(រយៈពេលជាន់ស្មាន ១២០ នាទី)

១. Custom Hook គឺជាអ្វី ១.១. Function ដែលចាប់ផ្តើមដោយ `use` និងផ្សំ Logic ដែលប្រើឡើងវិញបាន ២. របៀបបង្កើត Custom Hook ដោយខ្លួនឯង ២.១. ឧទាហរណ៍៖ `useFetch`, `useLocalStorage` ៣. សង្ខេបគ្រប់ Concept សំខាន់ៗ ៣.១. ពីមេរៀនទី១ ដល់ទី១៦ ៤. ការផ្សំ Component + State + Props + Effect + Router ចូលគ្នាជាគម្រោងពេញលេញ

លំហាត់អនុវត្ត (គម្រោងបញ្ចប់)

- បង្កើតកម្មវិធី **Todo App ពេញលេញ** ដែលមាន៖
  - Add / Delete / Mark as Done កិច្ចការ
  - រក្សាទុកទិន្នន័យតាមរយៈ Custom Hook
  - មាន ២ ទំព័រ៖ បញ្ជីកិច្ចការ និងទំព័របង្ហាញព័ត៌មានកម្មវិធី (About) ប្រើ React Router

---

សេចក្តីបញ្ចប់

- បន្ទាប់ពីបញ្ចប់ ១៧ មេរៀននេះ សិស្សនឹងមានសមត្ថភាព៖
  - បង្កើត Component, Style វា និងគ្រប់គ្រង State និង Props
  - ប្រើ Hook សំខាន់ៗ (useState, useEffect, useRef)
  - ប្រើ Portal សម្រាប់ Modal/Popup
  - ភ្ជាប់ API និងបង្កើតទំព័រច្រើនដោយ React Router
- ជំហានបន្ទាប់៖ សិក្សា State Management កម្រិតខ្ពស់ (Context API, Redux/Zustand) និង TypeScript ជាមួយ React